



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Student Skill Development Management System

AN ASSIGNMENT PROBLEM REPORT

*An Assignment Problem Report submitted in partial fulfilment of the
requirements for the Course of*

CSA4305 - INTERNET PROGRAMMING

to the award of the degree of

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE ENGINEERING**

Submitted by

Ebin Antony P (192373011)

Under the Supervision of

Guide

Dr. D. Sudhagar

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

SEPTEMBER 2026

Student Skill Development Management System (SSDMS)

Technologies

Course Outcome	Technology Used
CO1	HTML5, CSS3, JavaScript
CO2	XML, XSLT, JSP
Architecture	MVC
Database	MySQL
Server	Apache Tomcat
Development Tool	VS Code / Eclipse

The system will allow three types of users:

1. **Student**
2. **Mentor/Faculty**
3. **Administrator**

1. Main Features to Implement

Student Module

The student can:

- Register and log in.
- Create and edit a profile.
- Add technical and soft skills.
- Set proficiency levels.
- Set target skill levels.
- Take self-assessments.
- View skill gaps.
- Add certifications.
- Add projects.
- Add internship details.

- Track learning activities.
- View recommendations.
- Monitor progress.

Mentor Module

The mentor can:

- View student profiles.
- Review skills.
- Check assessment scores.
- Identify weak areas.
- Provide feedback.
- Recommend learning activities.

Administrator Module

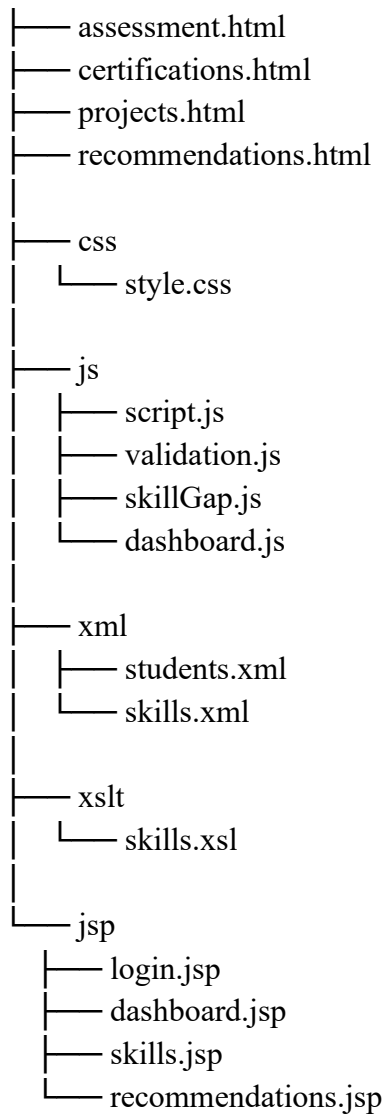
The administrator can:

- Manage students.
- Manage mentors.
- Manage skills.
- Manage learning resources.
- View analytics.
- Generate reports.

2. Suggested Project Folder Structure

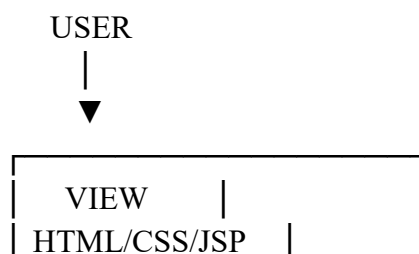
StudentSkillDevelopmentSystem

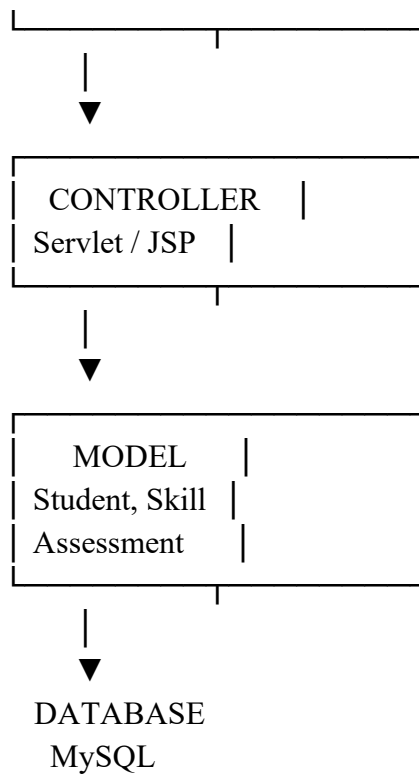
```
|  
|— index.html  
|— login.html  
|— register.html  
|— dashboard.html  
|— skills.html
```



This structure demonstrates modular and maintainable development, as required in the assignment.

3. System Architecture – MVC





MVC Components

Model

Responsible for data.

Examples:

- Student
- Skill
- Assessment
- Certification
- Project

View

Responsible for displaying information.

Examples:

- HTML
- CSS

- JSP
- XSLT output

Controller

Responsible for handling requests.

Examples:

- Login Controller
- Skill Controller
- Assessment Controller

4. Database Design

STUDENT

Student

student_id

name

email

password

department

year

SKILL_CATEGORY

Skill_Category

category_id

category_name

Examples:

- Programming
- Database
- Web Development

- Communication
- Soft Skills

SKILL

Skill

skill_id

skill_name

category_id

STUDENT_SKILL

Student_Skill

student_skill_id

student_id

skill_id

current_level

target_level

ASSESSMENT

Assessment

assessment_id

student_id

skill_id

score

assessment_date

CERTIFICATION

Certification

certificate_id

student_id

certificate_name

organization
completion_date
evidence

PROJECT

Project

project_id
student_id
project_name
description
technologies
status

MENTOR

Mentor

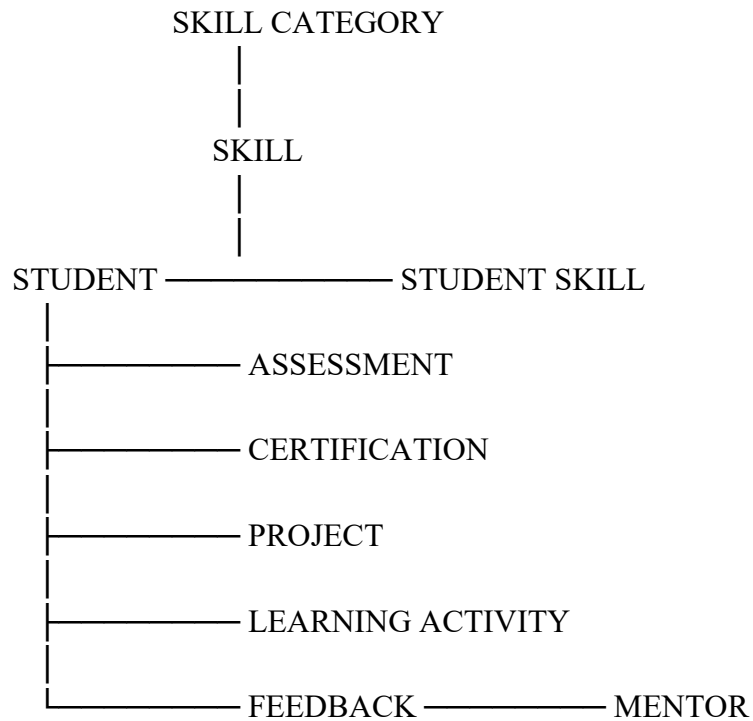
mentor_id
name
email
department

FEEDBACK

Feedback

feedback_id
mentor_id
student_id
comments
feedback_date

5. ER Diagram



6. Skill Gap Analysis

This is one of the important dynamic features of the project.

Formula

Skill Gap = Target Level – Current Level

Example

Skill	Current Level	Target Level	Gap
Python	2	5	3
Java	4	5	1
HTML	5	5	0

JavaScript Code

```
function calculateSkillGap(current, target) {  
  
    let gap = target - current;  
  
    if (gap <= 0) {  
        return "Target Achieved";  
    }  
  
    if (gap === 1) {  
        return "Minor Improvement Required";  
    }  
  
    if (gap === 2) {  
        return "Moderate Improvement Required";  
    }  
  
    return "Major Improvement Required";  
}
```

This directly demonstrates **CO1 dynamic functionality using JavaScript**.

7. Progress Calculation

Formula

Progress Percentage =
$$(\text{Current Skill Level} / \text{Target Skill Level}) \times 100$$

JavaScript

```
function calculateProgress(current, target) {  
  
    let progress = (current / target) * 100;  
  
    if (progress > 100) {
```

```
    progress = 100;
  }

  return progress.toFixed(2);
}
```

Example:

Current Level = 3

Target Level = 5

Progress = 60%

8. Recommendation Algorithm

A simple rule-based algorithm is suitable for this assignment.

```
function recommendLearning(level) {

  if (level <= 2) {
    return "Beginner Course and Basic Practice";
  }

  else if (level === 3) {
    return "Intermediate Course and Practice Exercises";
  }

  else if (level === 4) {
    return "Advanced Projects and Certification";
  }

  else {
    return "Industry-Level Projects and Mentoring";
  }
}
```

9. HTML Dashboard

```
<!DOCTYPE html>
<html>
<head>

  <title>Student Skill Dashboard</title>

  <link rel="stylesheet" href="css/style.css">

</head>

<body>

<div class="sidebar">

  <h2>SSDMS</h2>

  <a href="#">Dashboard</a>
  <a href="#">My Skills</a>
  <a href="#">Assessments</a>
  <a href="#">Certifications</a>
  <a href="#">Projects</a>
  <a href="#">Recommendations</a>

</div>

<div class="main">

  <h1>Student Dashboard</h1>

  <div class="cards">

    <div class="card">
      <h3>Total Skills</h3>
      <p id="totalSkills">5</p>
    </div>

    <div class="card">
```

```
        <h3>Certifications</h3>
        <p>3</p>
    </div>

    <div class="card">
        <h3>Projects</h3>
        <p>4</p>
    </div>

    <div class="card">
        <h3>Overall Progress</h3>
        <p>72%</p>
    </div>

</div>

</div>

<script src="js/dashboard.js"></script>

</body>
</html>
```

10. CSS Dashboard Design

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    display: flex;
    background: #f4f6f9;
}

.sidebar {
```

```
width: 220px;
min-height: 100vh;
background: #1e293b;
padding: 25px;
}
```

```
.sidebar h2 {
  color: white;
  margin-bottom: 30px;
}
```

```
.sidebar a {
  display: block;
  color: white;
  text-decoration: none;
  padding: 12px;
}
```

```
.sidebar a:hover {
  background: #334155;
}
```

```
.main {
  flex: 1;
  padding: 40px;
}
```

```
.cards {
  display: flex;
  gap: 20px;
  margin-top: 30px;
}
```

```
.card {
  background: white;
  padding: 25px;
  width: 200px;
  border-radius: 10px;
  box-shadow: 0px 3px 10px rgba(0,0,0,0.1);
}
```

```
.card p {  
  font-size: 28px;  
  margin-top: 15px;  
}
```

This section strongly demonstrates **CO1: visually appealing web pages using CSS**.

11. Student Skill Input Form

```
<h2>Add Skill</h2>
```

```
<form id="skillForm">
```

```
  <label>Skill Name</label>
```

```
  <input  
    type="text"  
    id="skillName"  
    placeholder="Enter Skill Name"  
    required  
  >
```

```
  <label>Current Level</label>
```

```
  <select id="currentLevel">
```

```
    <option value="1">1 - Beginner</option>  
    <option value="2">2 - Basic</option>  
    <option value="3">3 - Intermediate</option>  
    <option value="4">4 - Advanced</option>  
    <option value="5">5 - Expert</option>
```

```
  </select>
```

```
  <label>Target Level</label>
```

```
  <select id="targetLevel">
```



```
<option value="1">1</option>
<option value="2">2</option>
<option value="3">3</option>
<option value="4">4</option>
<option value="5">5</option>

</select>

<button type="submit">
  Add Skill
</button>

</form>
```

12. JavaScript Dynamic Skill Addition

```
document
.getElementById("skillForm")
.addEventListener("submit", function(event) {

  event.preventDefault();

  const skill =
    document.getElementById("skillName").value;

  const current =
    parseInt(
      document.getElementById("currentLevel").value
    );

  const target =
    parseInt(
      document.getElementById("targetLevel").value
    );

  const gap = target - current;
```

```
    alert(
        "Skill Added Successfully!\n" +
        "Skill: " + skill +
        "\nSkill Gap: " + gap
    );

});
```

13. XML Data Representation – CO2

Create students.xml.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<students>
```

```
    <student>
```

```
        <id>101</id>
```

```
        <name>Student Name</name>
```

```
        <department>CSE</department>
```

```
        <skills>
```

```
            <skill>
```

```
                <name>Python</name>
```

```
                <level>4</level>
```

```
            </skill>
```

```
            <skill>
```

```
                <name>Java</name>
```

```
<level>3</level>

</skill>

</skills>

</student>

</students>
```

14. XSLT – CO2

Create students.xsl.

```
<?xml version="1.0"?>

<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">

<xsl:template match="/">

<html>

<body>

<h2>Student Skill Report</h2>

<table border="1">

<tr>
<th>Skill</th>
<th>Level</th>
</tr>

<xsl:for-each
select="students/student/skills/skill">
```

```

<tr>

<td>
<xsl:value-of select="name"/>
</td>

<td>
<xsl:value-of select="level"/>
</td>

</tr>

</xsl:for-each>

</table>

</body>

</html>

</xsl:template>

</xsl:stylesheet>

```

This demonstrates the transformation of XML data into an HTML presentation.

15. JSP Implementation

Example dashboard.jsp:

```

<%@ page language="java" %>

<!DOCTYPE html>

<html>

<head>

```

```
<title>Student Dashboard</title>

</head>

<body>

<h1>
Welcome,
<%= request.getParameter("name") %>
</h1>

<p>
Student Skill Development Dashboard
</p>

</body>

</html>
```

16. MVC Implementation Example

Model

```
public class Skill {

    private String skillName;
    private int currentLevel;
    private int targetLevel;

    public String getSkillName() {
        return skillName;
    }

    public void setSkillName(String skillName) {
        this.skillName = skillName;
    }

    public int getCurrentLevel() {
```

```

        return currentLevel;
    }

    public void setCurrentLevel(int currentLevel) {
        this.currentLevel = currentLevel;
    }

    public int getTargetLevel() {
        return targetLevel;
    }

    public void setTargetLevel(int targetLevel) {
        this.targetLevel = targetLevel;
    }
}

```

Controller

```

@WebServlet("/skill")

public class SkillController
    extends HttpServlet {

    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response) {

        String skillName =
            request.getParameter("skillName");

        int currentLevel =
            Integer.parseInt(
                request.getParameter("currentLevel"));

        int targetLevel =
            Integer.parseInt(
                request.getParameter("targetLevel"));

        int skillGap =
            targetLevel - currentLevel;
    }
}

```

```
        request.setAttribute(  
            "skillGap",  
            skillGap  
        );  
    }  
}
```

View

```
<h2>Skill Gap Result</h2>
```

```
<p>
```

Your Skill Gap is:

```
${skillGap}
```

```
</p>
```

17. Testing and Validation

Test	Input	Expected Result
Registration	Valid student details	Student registered
Login	Valid credentials	Dashboard displayed
Add Skill	Python, Level 3	Skill added
Assessment	Score 80	Score stored
Skill Gap	Current 3, Target 5	Gap = 2
Recommendation	Level 2	Beginner course
Mentor Feedback	Feedback submitted	Feedback displayed

18. Performance Metrics

Assessment Improvement

Assessment Improvement =
Latest Score – Previous Score

Skill Gap Reduction

Skill Gap Reduction =
Previous Gap – Current Gap

Learning Completion Rate

Completion Rate =
Completed Activities / Total Activities $\times$ 100

19. Comparison of Two Approaches

Rule-Based System

Uses predefined conditions.

Example:

IF level $\leq$ 2
Recommend beginner course

Advantages

- Simple.
- Fast.
- Low cost.
- Easy to understand.

Disadvantages

- Less personalized.

Machine Learning Recommendation

Uses historical student data.

Advantages

- Personalized.
- Can discover patterns.

Disadvantages

- Requires large datasets.
- More complex.
- Higher development cost.

Selected Method

For this academic prototype, **Rule-Based Recommendation** is selected because it is transparent, easy to implement and does not require a large dataset.

20. Conclusion

The **Student Skill Development Management System** provides a centralized platform for tracking student skills, certifications, projects, assessments and learning progress.

The system satisfies **CO1** by using HTML, CSS and JavaScript to create an interactive and visually appealing web application. It satisfies **CO2** by using XML for structured data representation, XSLT for data presentation and JSP with MVC architecture for dynamic web application development.

The proposed system helps students identify skill gaps, receive learning recommendations and monitor progress. Mentors and administrators can use dashboards and reports to support evidence-based decisions related to student development and employability.